

Implementación y Análisis Experimental del Algoritmo de Gale–Shapley

Juan Pablo González-Mendoza
Departamento de Robótica y Mecatrónica
Centro de Investigación en Computación, IPN
Ciudad de México, México
jgonzaleza26@cic.mx

Abstract—En este trabajo se presenta la implementación y análisis experimental del algoritmo de emparejamiento estable de Gale–Shapley. Se desarrolla un generador de instancias sintéticas para evaluar el comportamiento del algoritmo bajo diferentes tamaños de entrada y modos de propuesta. Asimismo, se comparan implementaciones en Python y C++ con el objetivo de validar empíricamente la complejidad teórica $O(n^2)$ y analizar el impacto del lenguaje en el rendimiento computacional.

Index Terms—Emparejamiento estable, Gale–Shapley, complejidad computacional, análisis experimental, C++, Python.

I. INTRODUCCIÓN

El problema del emparejamiento estable, formulado por Gale y Shapley, consiste en encontrar una asignación entre dos conjuntos de igual tamaño tal que no exista un par que prefiera mutuamente emparejarse entre sí antes que con sus parejas actuales. Este problema posee aplicaciones en asignación de plazas médicas, sistemas de admisión y plataformas digitales de recomendación.

El objetivo de este trabajo es implementar el algoritmo clásico de Gale–Shapley, generar instancias controladas y analizar empíricamente su desempeño en Python y C++.

II. METODOLOGÍA

A. Generación de Instancias

Para garantizar reproducibilidad, se desarrolló un generador de instancias que produce perfiles de preferencias aleatorias. El usuario digita "m" para referirse a preferencia a los hombres y "w" a las mujeres. El orden de preferencia se escoge aleatoriamente como se mencionó antes.

Ejemplo de salida:

3
m
H0 M2 M0 M1
H1 M0 M2 M1
H2 M1 M0 M2
M0 H1 H0 H2
M1 H0 H2 H1
M2 H2 H1 H0

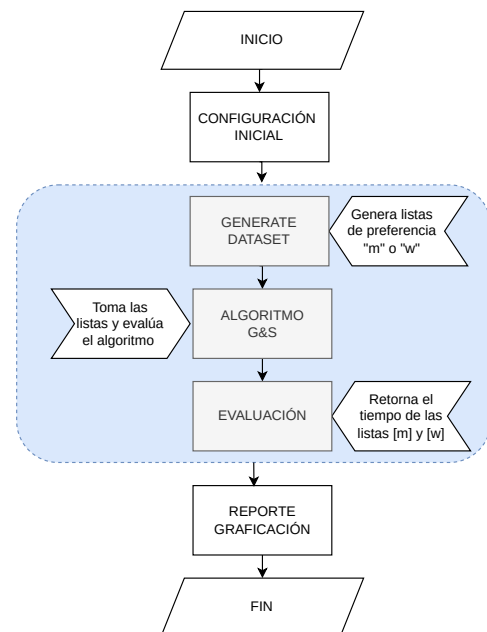


Fig. 1. Diseño de implementación del algoritmo Gale and Shapley

1

En es

B. Algoritmo de Gale–Shapley

TABLE I
EJECUCIÓN DEL ALGORITMO

Ronda	Propuesta	Resultado
1	H0 → M2	M2 libre ⇒ acepta. (H0,M2)
2	H1 → M0	M0 libre ⇒ acepta. (H1,M0)
3	H2 → M1	M1 libre ⇒ acepta. (H2,M1)
Resultado final: (H0,M2) (H1,M0) (H2,M1)		

III. RESULTADOS EXPERIMENTALES

Se realizaron pruebas para diferentes valores de n en ambos lenguajes. Las gráficas generadas muestran crecimiento cuadrático del tiempo de ejecución, confirmando la complejidad teórica.

Algorithm 1 Generación de Instancias de Emparejamiento

```

1: procedure GENERATEDATASET( $n, mode, s$ )
2:   Inicializar generador aleatorio  $s$ 
3:   Definir conjuntos  $H = \{H_0, \dots, H_{n-1}\}$  y  $W = \{W_0, \dots, W_{n-1}\}$ 
4:   Crear lista  $\mathcal{L}$  con  $n$  y  $mode$ 
5:   if  $mode = "m"$  then
6:     for all  $h \in H$  do
7:       Generar permutación aleatoria de  $W$ 
8:     end for
9:     for all  $w \in W$  do
10:      Generar permutación aleatoria de  $H$ 
11:    end for
12:   else
13:     for all  $w \in W$  do
14:       Generar permutación aleatoria de  $H$ 
15:     end for
16:     for all  $h \in H$  do
17:       Generar permutación aleatoria de  $W$ 
18:     end for
19:   end if
20:   Serializar y guardar en archivo .txt
21: end procedure

```

Algorithm 2 Algoritmo Gale-Shapley

```

1: procedure GALESHAPLEY( $n, Pref_P, Rank_R$ )
2:   Inicializar arreglos de emparejamiento
3:   Conjunto  $F$  de proponentes libres
4:   while  $F \neq \emptyset$  do
5:     Seleccionar  $p \in F$ 
6:      $r \leftarrow$  siguiente preferencia de  $p$ 
7:     if  $r$  está libre then
8:       Emparejar  $p$  con  $r$ 
9:     else
10:      Comparar rangos y actualizar emparejamiento
11:      if necesario then
12:        end if
13:      Retornar emparejamiento
14:    end while
15: end procedure

```

TABLE II
BENCHMARK ALGORITMO GALE-SHAPLEY - PYTHON
REPETICIONES: 3

n	H-proponen (s)	M-proponen (s)
5	0.023699	0.032120
10	0.024672	0.020448
20	0.031284	0.024579
50	0.024796	0.030515
100	0.031187	0.036778
200	0.059790	0.054848
300	0.103847	0.113359
500	0.272918	0.281324
750	0.587009	0.563379
1000	1.047477	1.048449

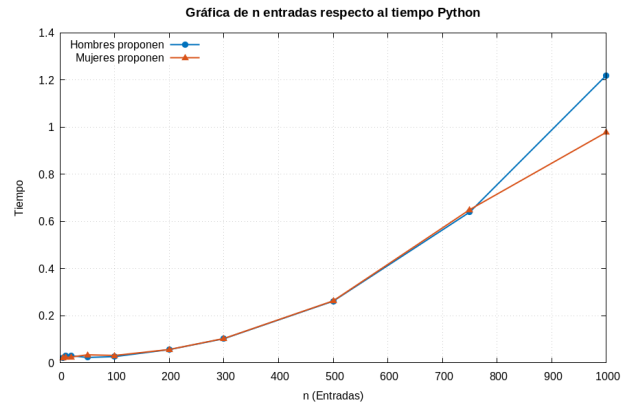


Fig. 2. Tiempo de ejecución en Python

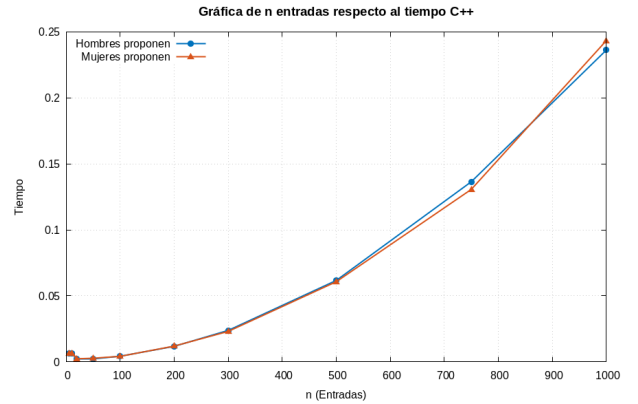


Fig. 3. Tiempo de ejecución en C++

TABLE III
BENCHMARK ALGORITMO GALE-SHAPLEY - C++
REPETICIONES: 3

n	H-proponen (s)	M-proponen (s)
5	0.002449	0.002837
10	0.004161	0.002323
20	0.004438	0.003813
50	0.003754	0.003109
100	0.007666	0.003685
200	0.013377	0.011594
300	0.026677	0.029002
500	0.070004	0.065064
750	0.148317	0.150865
1000	0.259636	0.263006

Los resultados muestran que C++ presenta una mejora significativa en tiempos de ejecución respecto a Python, especialmente para valores grandes de n .

IV. ANÁLISIS DE COMPLEJIDAD

El algoritmo garantiza terminación debido a que cada proponente avanza estrictamente en su lista de preferencias. En el peor caso, cada uno realiza n propuestas, resultando en una complejidad $O(n^2)$.

Asimismo, el emparejamiento obtenido es estable y óptimo para el grupo proponente.

TABLE IV
TIEMPOS DE EJECUCIÓN POR TAREA – GALE–SHAPLEY PYTHON
 $n = 500$, MODO: H-PROPONEN

Tarea	Tiempo (s)	(%)
Lectura stdin	0.001823	0.6
Init estructuras	0.000412	0.1
Lectura preferencias	0.089341	31.2
Conversión nombres→ID	0.124876	43.6
Gale–Shapley	0.065209	22.8
Salida	0.004812	1.7
TOTAL	0.286473	100.0

V. CONCLUSIONES

Este reporte presentó la implementación y comparación del algoritmo de emparejamiento estable en dos lenguajes de programación, evaluando su rendimiento mediante diversas entradas y etapas de procesamiento. El objetivo fue determinar la conveniencia del uso de cada lenguaje según el contexto de la aplicación, concluyendo que la elección depende de los requisitos de eficiencia; específicamente, para tareas que demandan procesamiento rápido de información, C++ ofrece ventajas significativas en tiempo de ejecución. Aunque la implementación en Python resultó más sencilla, C++ proporcionó un control más preciso sobre la gestión de memoria y la sintaxis, a pesar de los desafíos de configuración encontrados en la plataforma COACH. El desarrollo de este trabajo permitió validar la complejidad teórica mediante pruebas empíricas, reforzando la importancia de las estructuras de datos y el manejo adecuado de librerías para la medición de tiempos y generación de archivos de análisis.